

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



BUILD A SCROLLABLE LIST

Oleh:

Nazmi Hakim NIM. 2310817210012

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nazmi Hakim
NIM : 2310817210012

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Raka Azwar
NIM. 2210817210012

Andreyan Rizky Baskara, S.Kom., M.Kom.
NIP. 19930703 201903 01 011

DAFTAR ISI

LEMBAR PENGESAHAN	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1	6
A. Source Code.....	7
B. Output Program	20
C. Pembahasan	24
D. Tautan Git	34
SOAL 2.....	35
A. Pembahasan	35

DAFTAR GAMBAR

Gambar 1. Contoh UI List	6
Gambar 2. Contoh UI Detail.....	7
Gambar 3. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	20
Gambar 4. Screenshot Hasil Jawaban Soal 1 Jetpack Compose	21
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML.....	22
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML.....	23

DAFTAR TABEL

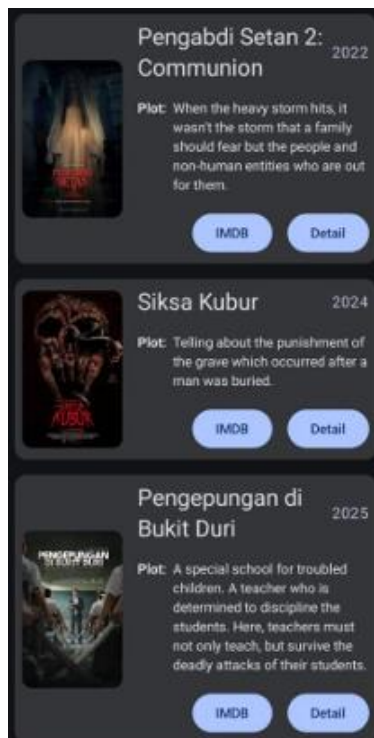
Tabel 1. Source Code Jawaban Soal 1 Jetpack Compose.....	7
Tabel 2. Source Code Jawaban Soal 1 Jetpack Compose.....	8
Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose.....	9
Tabel 4. Source Code Jawaban Soal 1 Jetpack Compose.....	11
Tabel 5. Source Code Jawaban Soal 1 XML.....	12
Tabel 6. Source Code Jawaban Soal 1 XML.....	12
Tabel 7. Source Code Jawaban Soal 1 XML.....	13
Tabel 8. Source Code Jawaban Soal 1 XML.....	14
Tabel 9. Source Code Jawaban Soal 1 XML.....	16
Tabel 10. Source Code Jawaban Soal 1 XML.....	16
Tabel 11. Source Code Jawaban Soal 1 XML.....	17
Tabel 12. Source Code Jawaban Soal 1 XML.....	18
Tabel 13. Source Code Jawaban Soal 1 XML.....	19
Tabel 14. Source Code Jawaban Soal 1 XML.....	19

SOAL 1

1. Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
 - a. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
 - b. Button kedua menggunakan Navigation component untuk membuka laman detail item
5. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
6. Saat orientasi perangkat berubah/dirotasi, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
7. Aplikasi menggunakan arsitektur single activity (satu activity memiliki beberapa fragment)
8. Aplikasi berbasis XML harus menggunakan ViewBinding

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut:



Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut:



Gambar 2. Contoh UI Detail

A. Source Code

Jetpack Compose :

MainActivity.kt :

Tabel 1. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.listcompose
2	
3	import android.os.Bundle
4	import androidx.activity.ComponentActivity
5	import androidx.activity.compose.setContent
6	import androidx.navigation.NavType
7	import androidx.navigation.compose.NavHost
8	import androidx.navigation.compose.composable
9	import androidx.navigation.compose.rememberNavController
10	import androidx.navigation.navArgument
11	import com.example.listcompose.ui.theme.ListComposeTheme
12	
13	class MainActivity : ComponentActivity() {
14	override fun onCreate(savedInstanceState: Bundle?) {
15	super.onCreate(savedInstanceState)

16	setContent {
17	ListComposeTheme {
18	val navController = rememberNavController()
19	NavHost(navController, startDestination =
20	"list") {
21	composable("list") {
22	ItemListScreen(navController)
23	}
24	composable(
25	"detail/{itemId}",
26	arguments =
27	listOf(navArgument("itemId") { type = NavType.IntType })
28	) { backStackEntry ->
29	val itemId =
30	backStackEntry.arguments?.getInt("itemId") ?: 0
31	DetailScreen(item =
32	ItemData.items[itemId])
33	}
34	}
35	}
	}

Item.kt :

Tabel 2. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.listcompose
2	
3	data class Item(
4	val id: Int,
5	val title: String,
6	val author: String,
7	val info1: String,
8	val info2: String,
9	val imageResId: Int,
10	val url: String
11	)
12	
13	object ItemData {
14	val items = listOf(
15	Item(0, "Lookism", "PTJ", "Action, Fighting, Comedy,
16	Romance", "Status: OnGoing", R.drawable.lookism,
17	"https://lookism.fandom.com/wiki/Lookism"),
18	Item(1, "Bones", "This, Picture: Mu Mu-min", "Action,
19	Fighting", "Status: OnGoing", R.drawable.bones,
20	"https://en.namu.wiki/w/%EB%B3%B8%EC%A6%88(%EC%9B%B9%ED%88%B0)",
21	Item(2, "WEE!!!", "Amoeba UwU", "Slice of Life, Romance",
	"OnGoing", R.drawable.wee,

	<pre> "https://webtoon.fandom.com/id/wiki/WEE!!!"), Item(3, "Hero Has Returned", "FUNGBACK", "Action, Fighting, Fantasy", "OnGoing", R.drawable.herohasreturned, "https://hero-has- returned.fandom.com/wiki/Hero_Has_Returned_Wiki"), Item(4, "Study Group", "Blue String", "Action, Fighting, Comedy", "OnGoing", R.drawable.studygroup, "https://study- group.fandom.com/wiki/Study_Group_Wiki")) } </pre>
--	--

ItemListScreen.kt :

Tabel 3. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.listcompose
2	
3	import android.content.Intent
4	import android.net.Uri
5	import androidx.compose.foundation.Image
6	import androidx.compose.foundation.layout.Arrangement
7	import androidx.compose.foundation.layout.Column
8	import androidx.compose.foundation.layout.PaddingValues
9	import androidx.compose.foundation.layout.Row
10	import androidx.compose.foundation.layout.Spacer
11	import androidx.compose.foundation.layout.fillMaxWidth
12	import androidx.compose.foundation.layout.height
13	import androidx.compose.foundation.layout.padding
14	import androidx.compose.foundation.lazy.LazyColumn
15	import androidx.compose.foundation.shape.RoundedCornerShape
16	import androidx.compose.material3.Button
17	import androidx.compose.material3.Card
18	import androidx.compose.material3.CardDefaults
19	import androidx.compose.material3.MaterialTheme
20	import androidx.compose.material3.Text
21	import androidx.compose.runtime.Composable
22	import androidx.compose.ui.Modifier
23	import androidx.compose.ui.draw.clip
24	import androidx.compose.ui.graphics.Color
25	import androidx.compose.ui.layout.ContentScale
26	import androidx.compose.ui.platform.LocalContext
27	import androidx.compose.ui.res.painterResource
28	import androidx.compose.ui.unit.dp
29	import androidx.navigation.NavController
30	
31	@Composable
32	fun ItemListScreen(navController: NavController) {
33	LazyColumn(contentPadding = PaddingValues(8.dp)) {
34	items(ItemData.items.size) { index ->
35	val item = ItemData.items[index]

```

36         Card(
37             shape = RoundedCornerShape(16.dp) ,
38             modifier = Modifier
39                 .padding(8.dp)
40                 .fillMaxWidth() ,
41             colors = CardDefaults.cardColors(containerColor
42 = Color.DarkGray) ,
43             elevation =
44 CardDefaults.cardElevation(defaultElevation = 4.dp)
45         ) {
46             Column(modifier = Modifier.padding(16.dp)) {
47                 Image(
48                     painter = painterResource(id =
49 item.imageResId) ,
50                     contentDescription = item.title ,
51                     modifier = Modifier
52                         .fillMaxWidth()
53                         .height(180.dp)
54                         .clip(RoundedCornerShape(16.dp)) ,
55                     contentScale = ContentScale.Crop
56                 )
57                 Spacer(modifier = Modifier.height(8.dp))
58                 Text(text = item.title, style =
59 MaterialTheme.typography.titleLarge, color = Color.White)
60                 Text(text = item.author, style =
61 MaterialTheme.typography.bodyMedium, color = Color.White)
62                 Text(text = item.info1, style =
63 MaterialTheme.typography.bodySmall, color = Color.White)
64                 Text(text = item.info2, style =
65 MaterialTheme.typography.bodySmall, color = Color.White)
66                 Spacer(modifier = Modifier.height(8.dp))
67                 Row(horizontalArrangement =
68 Arrangement.spacedBy(8.dp)) {
69                     val context = LocalContext.current
70                     Button(onClick = {
71                         val intent =
72 Intent(Intent.ACTION_VIEW, Uri.parse(item.url))
73                         context.startActivity(intent)
74                     }) {
75                         Text("Lihat Web")
76                     }
77                     Button(onClick = {
78 navController.navigate("detail/${item.id}")
79                         }) {
80                         Text("Detail")
81                     }
82                 }
83             }
84         }
85     }
86 }

```

	<pre> } } </pre>
--	------------------------------

DetailScreen.kt :

Tabel 4. Source Code Jawaban Soal 1 Jetpack Compose

1	package com.example.listcompose
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.Composable
8	import androidx.compose.ui.Modifier
9	import androidx.compose.ui.draw.clip
10	import androidx.compose.ui.layout.ContentScale
11	import androidx.compose.ui.res.painterResource
12	import androidx.compose.ui.unit.dp
13	
14	@Composable
15	fun DetailScreen(item: Item) {
16	Column(modifier = Modifier.padding(16.dp)) {
17	Image(
18	painter = painterResource(id = item.imageResId),
19	contentDescription = item.title,
20	modifier = Modifier
21	.fillMaxWidth()
22	.height(240.dp)
23	.clip(RoundedCornerShape(16.dp)),
24	contentScale = ContentScale.Crop
25	)
26	Spacer(modifier = Modifier.height(16.dp))
27	Text(text = item.title, style =
28	MaterialTheme.typography.headlineMedium)
29	Text(text = item.author, style =
30	MaterialTheme.typography.bodyLarge)
31	Text(text = item.info1, style =
32	MaterialTheme.typography.bodyMedium)
	Text(text = item.info2, style =
	MaterialTheme.typography.bodyMedium)
	}
	}

XML :

Item.kt :

Tabel 5. Source Code Jawaban Soal 1 XML

1	package	com.example.listxml
2		
3	data	class Item(
4	val	imageResId: Int,
5	val	title: String,
6	val	author: String,
7	val url: String)	

ItemAdapter.kt :

Tabel 6. Source Code Jawaban Soal 1 XML

1	package com.example.listxml
2	
3	import android.view.LayoutInflater
4	import android.view.ViewGroup
5	import androidx.recyclerview.widget.RecyclerView
6	import com.example.listxml.databinding.ItemListBinding
7	
8	class ItemAdapter(
9	private val items: List<Item>,
10	private val onClick: (String) -> Unit,
11	private val onDetailClick: (Item) -> Unit
12	) : RecyclerView.Adapter<ItemAdapter.ItemViewHolder>() {
13	
14	inner class ItemViewHolder(val binding: ItemListBinding) :
15	RecyclerView.ViewHolder(binding.root)
16	
17	override fun onCreateViewHolder(parent: ViewGroup,
18	viewType: Int): ItemViewHolder {
19	val binding =
20	ItemListBinding.inflate(LayoutInflater.from(parent.context),
21	parent, false)
22	return ItemViewHolder(binding)
23	}
24	
25	override fun onBindViewHolder(holder: ItemViewHolder,
26	position: Int) {
27	val item = items[position]
28	with(holder.binding) {
29	itemImage.setImageResource(item.imageResId)
30	titleText.text = item.title
31	subtitleText.text = item.author
32	
33	openButton.setOnClickListener {

34	onOpenClick(item.url)
35	}
36	detailButton.setOnClickListener {
37	onDetailClick(item)
38	}
39	}
	}
	override fun getItemCount(): Int = items.size
	}

item_list.xml :

Tabel 7. Source Code Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.cardview.widget.CardView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:card_view="http://schemas.android.com/apk/res-auto"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:layout_width="match_parent"
7	android:layout_height="wrap_content"
8	android:layout_marginBottom="12dp"
9	card_view:cardCornerRadius="16dp"
10	card_view:cardElevation="4dp">
11	
12	<LinearLayout
13	android:layout_width="match_parent"
14	android:layout_height="wrap_content"
15	android:orientation="vertical"
16	android:padding="16dp">
17	
18	
19	<com.google.android.material.imageview.ShapeableImageView
20	android:id="@+id/itemImage"
21	android:layout_width="match_parent"
22	android:layout_height="180dp"
23	android:scaleType="centerCrop"
24	
25	app:shapeAppearanceOverlay="@style/RoundedImageView"/>
26	
27	<TextView
28	android:id="@+id/titleText"
29	android:layout_width="wrap_content"
30	android:layout_height="wrap_content"
31	android:text="Judul"
32	android:textStyle="bold"
33	android:textSize="18sp"
34	android:paddingTop="8dp"/>
35	
36	<TextView
37	android:id="@+id/subtitleText"

38	android:layout_width="wrap_content"	
39	android:layout_height="wrap_content"	
40	android:text="Subjudul"	/>
41		
42	<LinearLayout	
43	android:layout_width="match_parent"	
44	android:layout_height="wrap_content"	
45	android:orientation="horizontal"	
46	android:gravity="end">	
47		
48	<Button	
49	android:id="@+id/openButton"	
50	android:layout_width="wrap_content"	
51	android:layout_height="wrap_content"	
52	android:text="Open"	/>
53		
54	<Button	
55	android:id="@+id/detailButton"	
56	android:layout_width="wrap_content"	
57	android:layout_height="wrap_content"	
58	android:text="Detail"	
59	android:layout_marginStart="8dp"/>	
60	</LinearLayout>	
	</LinearLayout>	
	</androidx.cardview.widget.CardView>	

HomeFragment.kt :

Tabel 8. Source Code Jawaban Soal 1 XML

1	package com.example.listxml
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.os.Bundle
6	import android.view.LayoutInflater
7	import android.view.View
8	import android.view.ViewGroup
9	import androidx.fragment.app.Fragment
10	import androidx.navigation.fragment.findNavController
11	import androidx.recyclerview.widget.LinearLayoutManager
12	import com.example.listxml.databinding.FragmentHomeBinding
13	
14	class HomeFragment : Fragment() {
15	private var _binding: FragmentHomeBinding? = null
16	private val binding get() = _binding!!
17	
18	override fun onCreateView(
19	inflater: LayoutInflater, container: ViewGroup?,
20	savedInstanceState: Bundle?

```

21     ): View {
22         _binding = FragmentHomeBinding.inflate(inflater,
23 container, false)
24         return binding.root
25     }
26
27     override fun onCreateView(view: View, savedInstanceState:
28 Bundle?) {
29         val itemList = listOf(
30             Item(R.drawable.lookism, "Lookism", "PTJ",
31 "https://example.com"),
32             Item(R.drawable.bones, "Bones", "This, Picture : Mu
33 Mu-min", "https://example.com"),
34             Item(R.drawable.wee, "WEE!!!", "Amoeba UwU",
35 "https://example.com"),
36             Item(R.drawable.herohasreturned, "Hero Has
37 Returned", "FUNGBACK", "https://example.com"),
38             Item(R.drawable.studygroup, "Study Group",
39 "BlueString", "https://example.com")
40         )
41
42         val adapter = ItemAdapter(itemList,
43             onClick = { url ->
44                 val intent = Intent(Intent.ACTION_VIEW,
45 Uri.parse(url))
46                 startActivity(intent)
47             },
48             onDetailClick = { item ->
49                 val action =
50 HomeFragmentDirections.actionHomeToDetail(item.title,
51 item.author, item.imageResId)
52                 findNavController().navigate(action)
53             })
54
55         binding.recyclerView.layoutManager =
56 LinearLayoutManager(requireContext())
57         binding.recyclerView.adapter = adapter
58     }
59
60     override fun onDestroyView() {
61         super.onDestroyView()
62         _binding = null
63     }
64 }

```

fragment_home.xml :

Tabel 9. Source Code Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.constraintlayout.widget.ConstraintLayout
3	
4	xmlns:android="http://schemas.android.com/apk/res/android"
5	xmlns:app="http://schemas.android.com/apk/res-auto"
6	android:layout_width="match_parent"
7	android:layout_height="match_parent">
8	
9	<androidx.recyclerview.widget.RecyclerView
10	android:id="@+id/recyclerView"
11	android:layout_width="0dp"
12	android:layout_height="0dp"
13	android:clipToPadding="false"
14	android:padding="16dp"
15	app:layoutManager=
16	"androidx.recyclerview.widget.LinearLayoutManager"
17	app:layout_constraintTop_toTopOf="parent"
18	app:layout_constraintBottom_toBottomOf="parent"
19	app:layout_constraintStart_toStartOf="parent"
	app:layout_constraintEnd_toEndOf="parent"/>
	</androidx.constraintlayout.widget.ConstraintLayout>

nav_graph :

Tabel 10. Source Code Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	xmlns:tools="http://schemas.android.com/tools"
6	android:id="@+id/nav_graph"
7	app:startDestination="@id/homeFragment">
8	
9	<fragment
10	android:id="@+id/homeFragment"
11	android:name="com.example.listxml.HomeFragment"
12	android:label="Home"
13	tools:layout="@layout/fragment_home">
14	<action
15	android:id="@+id/action_home_to_detail"
16	app:destination="@id/detailFragment" />
17	</fragment>
18	
19	<fragment
20	android:id="@+id/detailFragment"
21	android:name="com.example.listxml.DetailFragment"
22	android:label="Detail"

23	tools:layout="@layout/fragment_detail">
24	<argument
25	android:name="title"
26	app:argType="string" />
27	<argument
28	android:name="subtitle"
29	app:argType="string" />
30	<argument
31	android:name="imageResId"
32	app:argType="integer" />
33	</fragment>
	</navigation>

DetailFragment.kt :

Tabel 11. Source Code Jawaban Soal 1 XML

1	package com.example.listxml
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import androidx.navigation.fragment.navArgs
9	import com.example.listxml.databinding.FragmentDetailBinding
10	
11	class DetailFragment : Fragment() {
12	private var _binding: FragmentDetailBinding? = null
13	private val binding get() = _binding!!
14	private val args: DetailFragmentArgs by navArgs()
15	
16	override fun onCreateView(
17	inflater: LayoutInflater, container: ViewGroup?,
18	savedInstanceState: Bundle?
19	): View {
20	_binding = FragmentDetailBinding.inflate(inflater,
21	container, false)
22	return binding.root
23	}
24	
25	override fun onViewCreated(view: View, savedInstanceState:
26	Bundle?) {
27	binding.detailTitle.text = args.title
28	binding.detailSubtitle.text = args.subtitle
29	binding.detailImage.setImageResource(args.imageResId)
30	}
31	
32	override fun onDestroyView() {
33	super.onDestroyView()

34	<pre> _binding = null } }</pre>
----	---

Fragment_detail.xml:

Tabel 12. Source Code Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<ScrollView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:padding="16dp">
8	
9	<LinearLayout
10	android:layout_width="match_parent"
11	android:layout_height="wrap_content"
12	android:orientation="vertical">
13	
14	
15	<com.google.android.material.imageview.ShapeableImageView
16	android:id="@+id/detailImage"
17	android:layout_width="match_parent"
18	android:layout_height="200dp"
19	android:scaleType="centerCrop"
20	
21	app:shapeAppearanceOverlay="@style/RoundedImageView" />
22	
23	<TextView
24	android:id="@+id/detailTitle"
25	android:layout_width="wrap_content"
26	android:layout_height="wrap_content"
27	android:textStyle="bold"
28	android:textSize="20sp"
29	android:paddingTop="8dp" />
30	
31	<TextView
32	android:id="@+id/detailSubtitle"
33	android:layout_width="wrap_content"
	android:layout_height="wrap_content" />
	</LinearLayout>
	</ScrollView>

MainActivity.kt:

Tabel 13. Source Code Jawaban Soal 1 XML

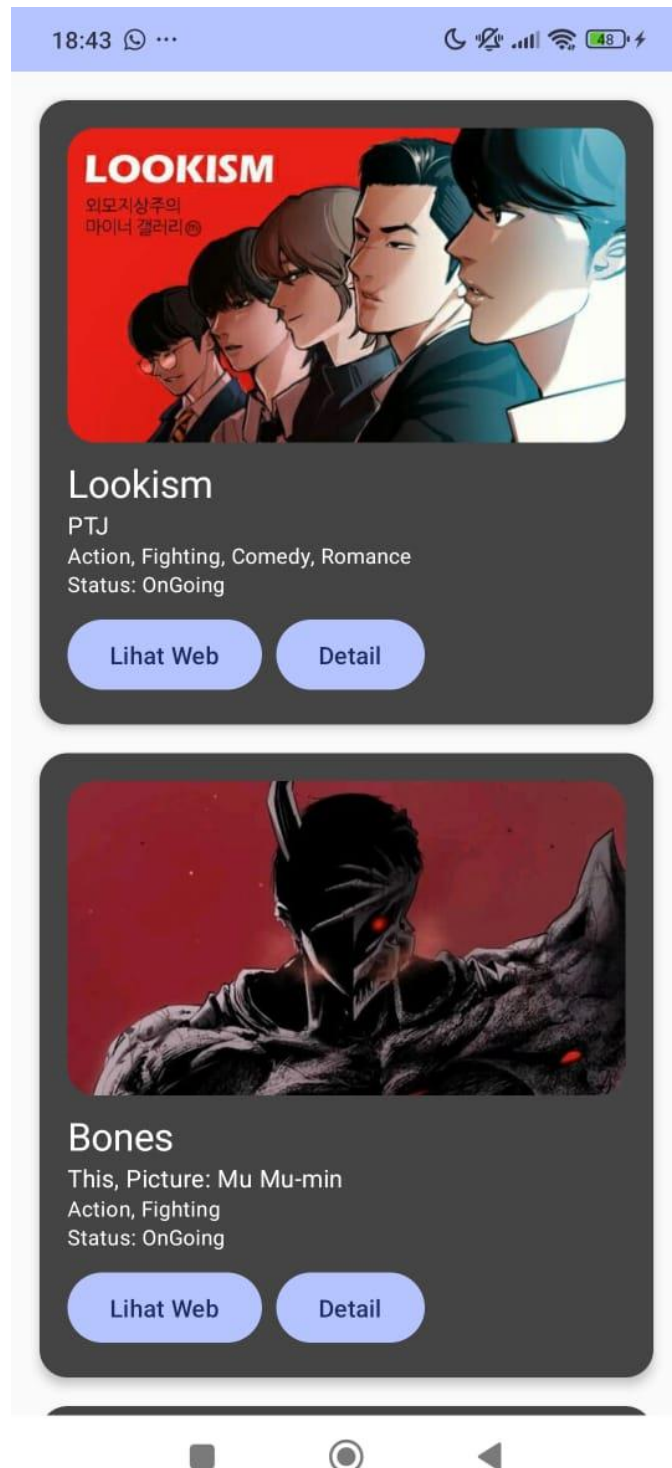
1	package com.example.listxml
2	
3	import android.os.Bundle
4	import androidx.appcompat.app.AppCompatActivity
5	import com.example.listxml.databinding.ActivityMainBinding
6	
7	class MainActivity : AppCompatActivity() {
8	private lateinit var binding: ActivityMainBinding
9	
10	override fun onCreate(savedInstanceState: Bundle?) {
11	super.onCreate(savedInstanceState)
12	binding = ActivityMainBinding.inflate(layoutInflater)
13	setContentView(binding.root)
14	}
15	}

activity_main.xml:

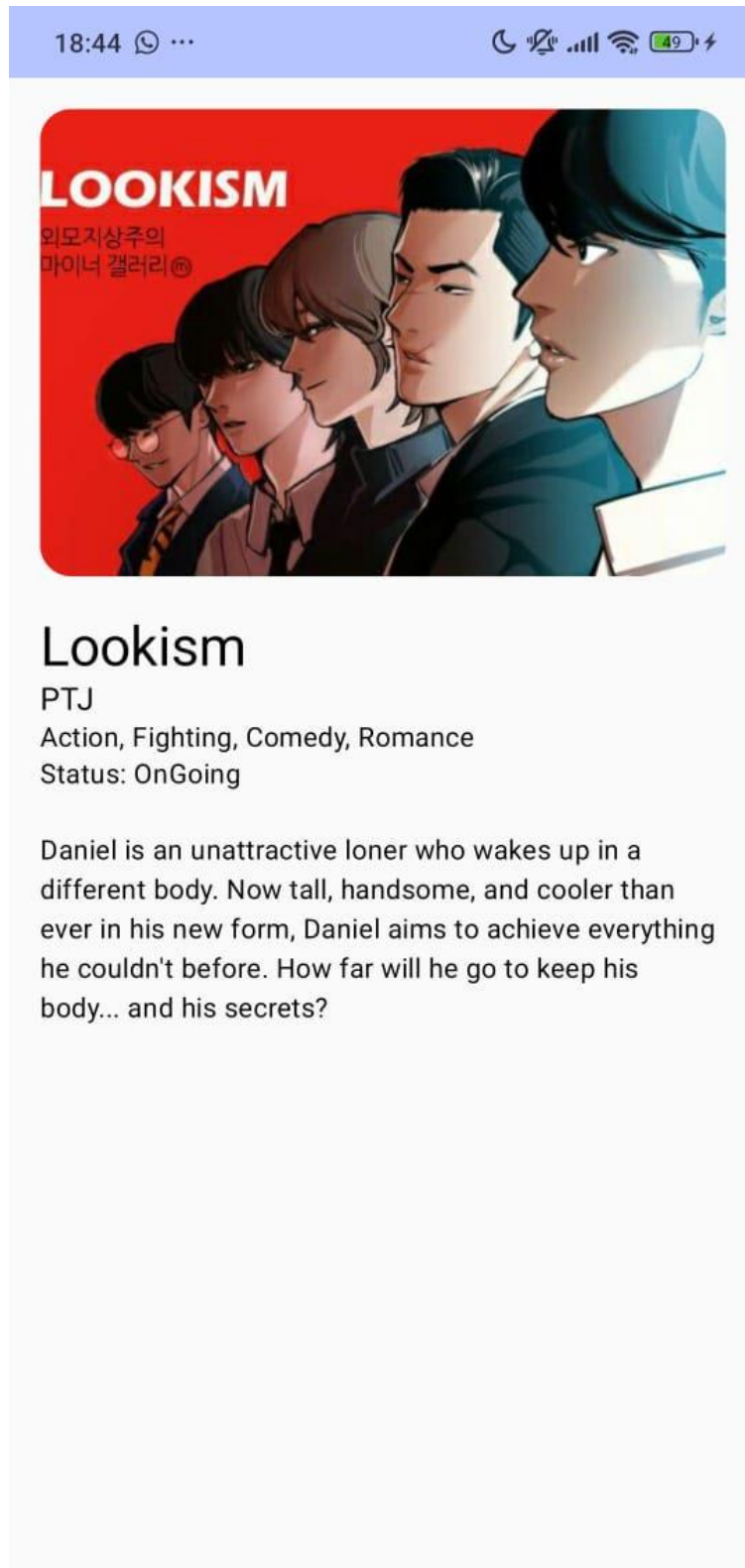
Tabel 14. Source Code Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<androidx.fragment.app.FragmentContainerView
3	xmlns:android="http://schemas.android.com/apk/res/android"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:id="@+id/nav_host_fragment"
6	android:name="androidx.navigation.fragment.NavHostFragment"
7	android:layout_width="match_parent"
8	android:layout_height="match_parent"
9	app:navGraph="@navigation/nav_graph"
10	app:defaultNavHost="true" />

B. Output Program
Compose :

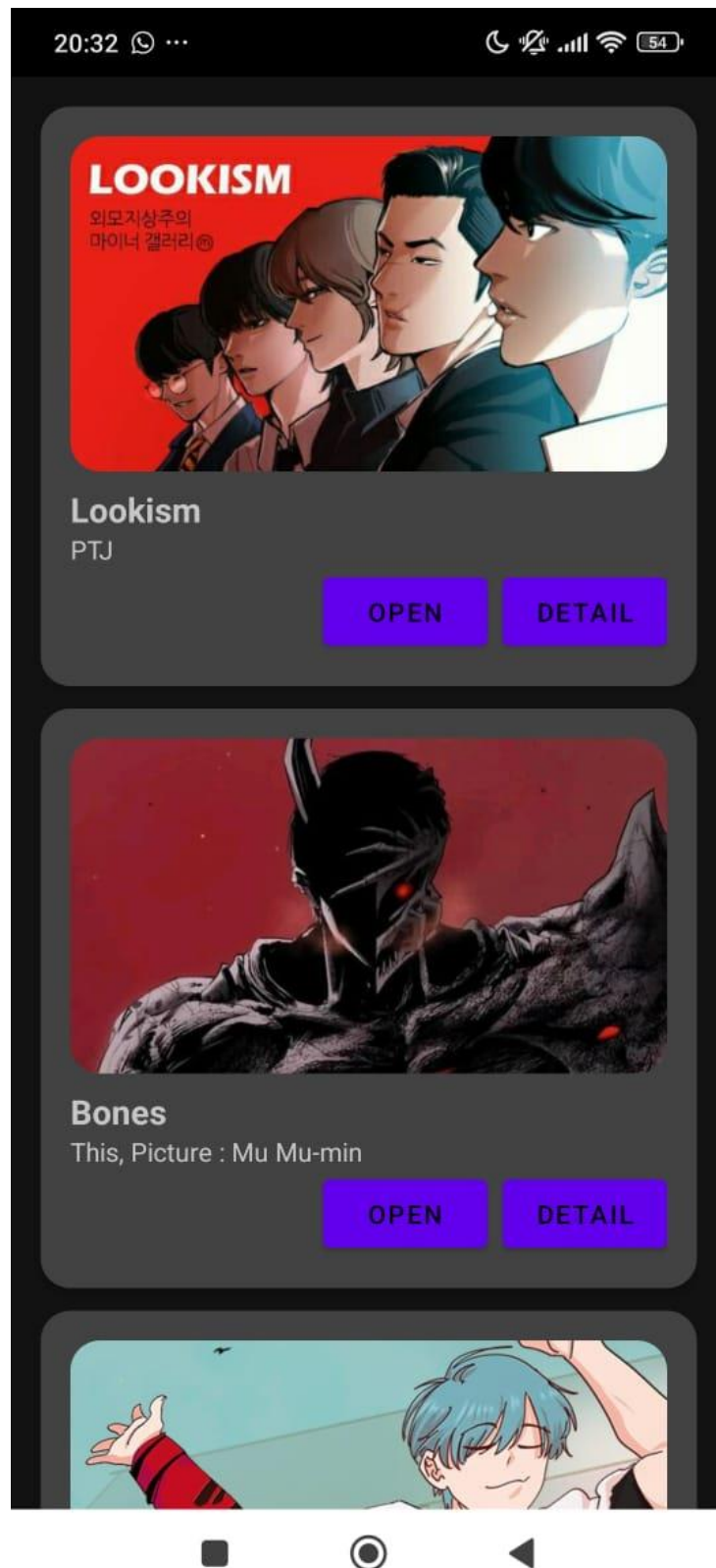


Gambar 3. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

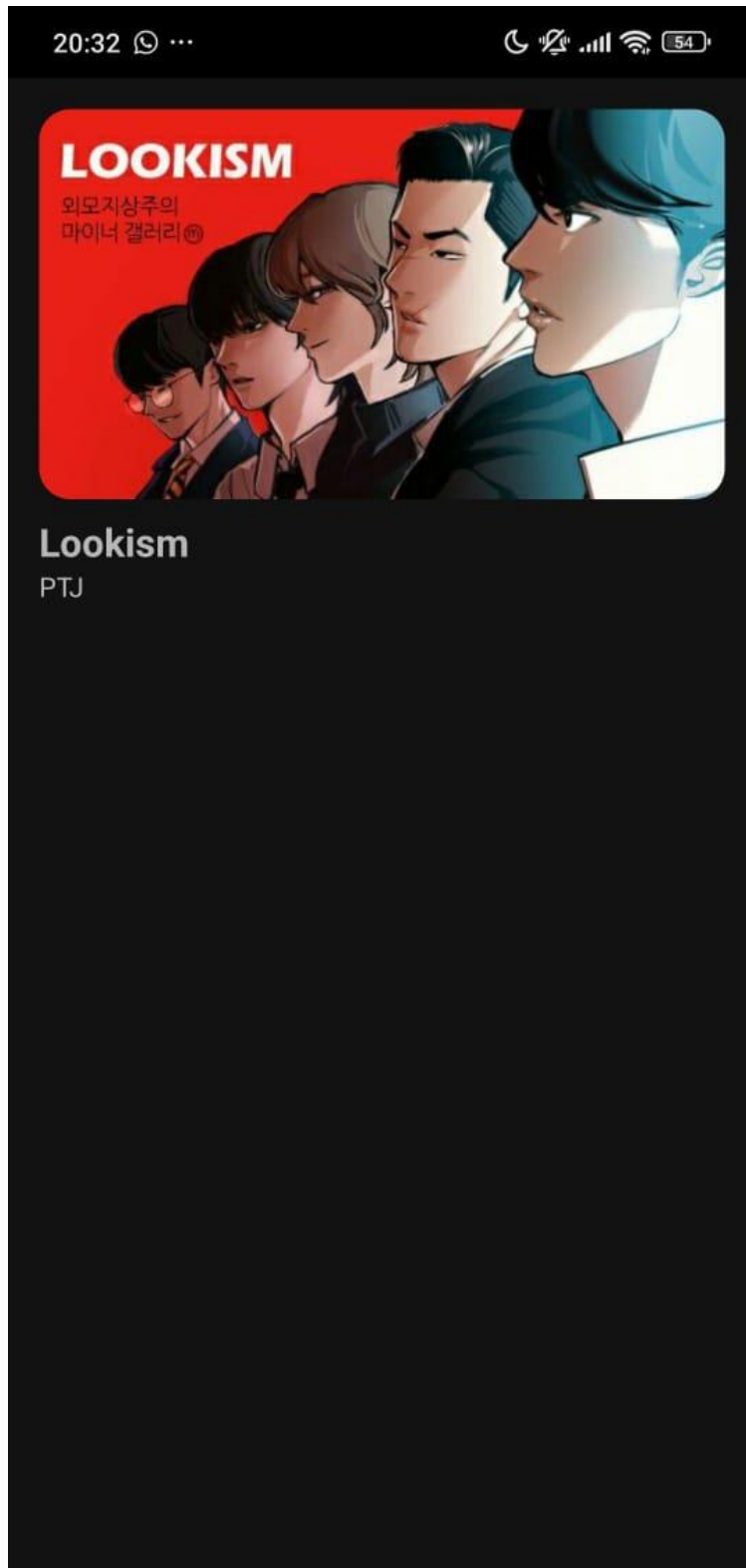


Gambar 4. Screenshot Hasil Jawaban Soal 1 Jetpack Compose

XML :



Gambar 5. Screenshot Hasil Jawaban Soal 1 XML



Gambar 6. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

Jetpack Compose :

MainActivity.kt:

Line 18, Membuat dan menyimpan instance NavController (digunakan untuk navigasi antar layar). rememberNavController() memastikan controller tetap konsisten meskipun Compose melakukan recomposition (UI redraw).

Line 19, NavHost adalah **container navigasi Compose** yang menentukan layar mana yang akan ditampilkan berdasarkan route yang aktif. startDestination = "list" artinya layar pertama yang ditampilkan adalah dengan route "list".

Line 20 sampai 22, Mendefinisikan route "list", yaitu layar utama yang menampilkan daftar item. Saat route ini aktif, akan dipanggil fungsi: "ItemListScreen(navController)" yang artinya akan menampilkan list (ItemListScreen.kt) dan membutuhkan NavController untuk navigasi saat tombol "Detail" ditekan.

Line 23 sampai 24, adalah rute untuk layar detail dengan parameter itemId sebagai bagian dari route. ItemId diambil dari Index di ItemListScreen

Line 25, Mendefinisikan bahwa itemId adalah parameter bertipe integer. Compose akan otomatis mengurai argumen tersebut dari string ke int saat route ini dipanggil.

Line 26 sampai 28, backStackEntry berisi semua informasi tentang argumen dan status navigasi saat ini. "val itemId = backStackEntry.arguments?.getInt("itemId") ?: 0" Mengambil nilai itemId dari argumen. Jika tidak ada, default ke 0. Lalu memanggil: DetailScreen(item = ItemData.items[itemId]) untuk menampilkan UI detail berdasarkan data item di itemId.

Item.kt :

Line 3 sampai 12, Ini mendeklarasikan **data class** bernama Item, yaitu blueprint untuk satu item yang akan ditampilkan di daftar (list) :

val id: Int, ID unik untuk tiap item. Bisa digunakan untuk indexing, navigasi, atau pembeda.

val title: String, judul dari item (misalnya judul webtoon atau komik).

val author: String, nama pembuat atau pengarang.

val info1: String, informasi tambahan 1, seperti genre.

val info2: String, informasi tambahan 2, seperti status (OnGoing, Completed).

val imageResId: Int, ID resource gambar (R.drawable.nama_gambar) yang mereferensikan gambar di folder res/drawable.

val url: String, URL website atau sumber informasi item.

Line 13 sampai 21, Membuat object bernama ItemData yang berisi sebuah variabel items, yaitu list yang berisi beberapa objek dari data class Item. Setiap item memiliki data seperti ID, judul, penulis, genre, status, gambar, dan URL. Data ini nantinya digunakan untuk ditampilkan pada list dan detail screen.

DetailScreen.kt :

Line 3 sampai 14, Import berbagai fungsi dan kelas dari Jetpack Compose yang digunakan untuk membangun UI. Image, Column, Spacer, Text merupakan elemen UI Compose. painterResource untuk memuat gambar dari res/drawable. MaterialTheme mengambil gaya dari tema Material 3. Modifier dan dp untuk styling dan ukuran layout.

Line 16 sampai 17, @Composable memberitahu bahwa fungsi ini bisa digunakan untuk menyusun UI dalam Jetpack Compose. DetailScreen menerima parameter item bertipe Item, yang berisi data seperti gambar, judul, author, dll.

Line 18, mengelola dan menyimpan posisi scroll secara stateful pada layout yang bisa digulir.

Line 19 sampai 24, membuat layout vertical yang menempatkan elemen dari atas ke bawah, dan memberikan jarak di sekelilingnya sebesar 16dp

Line 25 sampai 32, digunakan untuk menampilkan gambar utama dari item, dengan pengaturan sebagai berikut: gambar dimuat menggunakan painterResource berdasarkan ID gambar (item.imageResId) yang berada di folder res/drawable, dan diberikan contentDescription berupa judul item untuk mendukung aksesibilitas (dibaca oleh screen reader). Gambar menggunakan Modifier agar memenuhi lebar layar (fillMaxWidth()), memiliki tinggi 240dp (height(240.dp)), dan sudutnya dibulatkan menggunakan clip(RoundedCornerShape(16.dp)). Selain itu, contentScale = ContentScale.Crop digunakan untuk memotong gambar agar pas dalam ukuran tersebut tanpa mengubah rasio aslinya.

Line 34, spacer untuk membuat jarak setinggi 16dp

Line 35 sampai 39, menampilkan informasi dari item secara berurutan, dimulai dari item.title sebagai judul dengan gaya teks besar, diikuti oleh item.author yang menunjukkan nama pengarang, lalu item.info1 sebagai genre atau kategori, dan terakhir item.info2 yang berisi status seperti "OnGoing"; setiap teks ini menggunakan gaya dari MaterialTheme.typography yang mengikuti tampilan standar Material Design 3.

ItemListScreen.kt :

Line 3 sampai 29, Mengimpor berbagai komponen Compose, termasuk layout (Column, Row, LazyColumn, dll.), Material 3 (Card, Button, dll.), navigasi (NavController), dan Android Intent untuk membuka URL.

Line 31 dan 32, Fungsi composable utama yang menampilkan daftar item, navController digunakan untuk navigasi antar layar.

Line 33, Membuat membuat komponen tata letak vertikal yang bisa di scroll (mirip RecyclerView) dengan padding di sekeliling isi sebesar 8dp.

Line 34, melakukan loop sebanyak ItemData.items (saat ini ada 5) dan index adalah index item saat ini

Line 35, Mengambil item berdasarkan indeks.

Line 36 sampai 43, Membungkus setiap item dalam sebuah Card dengan sudut bulat (RoundedCornerShape(16.dp)), Padding 8dp, mengisi lebar sepenuh layer (di kode ini dikurangi oleh padding 8dp, warna latar gelap (DarkGray), dan elevasi bayangan 4dp.

Line 44, Menyusun isi card secara vertikal dengan padding dalam sebesar 16dp.

Line 45 sampai 53, Menampilkan gambar dari res/drawable berdasarkan item.imageResId. Lebar penuh (dikurangi padding 16dp), tinggi 180dp, sudut bulat. contentScale = Crop, agar gambar disesuaikan supaya penuh dan proporsional.

Line 54, Menambahkan jarak vertical setinggi 8dp.

Line 55 sampai 58, Menampilkan informasi item.title, item.author, item.info1, item.info2 yang diambil dari item.kt dengan berbagai gaya teks dan warna putih.

Line 59, Menambahkan jarak vertical setinggi 8dp.

Line 60, Row menyusun tombol horizontal dengan jarak antar tombol 8dp.

Line 61, Mengakses Context Android lokal untuk membuka URL menggunakan Intent.

Line 62 sampai 65, Membuat button dengan teks "Lihat Web". Saat diklik, membuka browser dan menampilkan URL dari item menggunakan Intent.ACTION_VIEW.

Line 68 sampai 71, Saat diklik, navigasi ke halaman detail item berdasarkan item.id (0 sampai 4)

XML :

Item.kt :

Line 3, sebuah data class bernama Item

Line 4 sampai 7, merupakan objek-objek immutable (val) yang menyimpan id dari setiap gambar item (imageResId: Int), title atau judul item (title: String), nama pembuat, penulis, atau deskripsi tambahan. (author: String), dan URL yang akan dibuka jika tombol "Open" diklik. Biasanya berupa link web atau tautan aplikasi (url: String)

ItemAdapter.kt :

Line 3 sampai 6, Berisikan import yang dibutuhkan :

LayoutInflater: untuk mengubah file XML layout menjadi objek View.

ViewGroup: induk dari layout.

RecyclerView: komponen utama untuk list.

ItemListAdapter: class binding otomatis yang dihasilkan dari file item_list.xml.

Line 8 sampai 12, Adapter menerima sebuah list data (items) bertipe List<Item> untuk ditampilkan, serta dua lambda function yaitu onItemClick bertipe (String) -> Unit yang menangani klik tombol "Open" dan menerima URL sebagai parameter, dan onItemClick bertipe (Item) -> Unit yang menangani klik tombol "Detail" dengan membawa seluruh data Item sebagai parameter. Unit dalam Kotlin setara dengan void di Java, artinya fungsi tersebut tidak mengembalikan nilai apa pun; adapter ini mewarisi dari RecyclerView.Adapter dan menggunakan ViewHolder sebagai kelas ViewHolder-nya.

Line 14, mendefinisikan kelas ViewHolder sebagai tempat menyimpan referensi ke layout per item (item_list.xml) secara efisien menggunakan ViewBinding, dan mewarisi RecyclerView.ViewHolder agar dapat digunakan oleh adapter untuk menampilkan data dalam RecyclerView.

Line 16 sampai 19, onCreateViewHolder dipanggil oleh RecyclerView saat membutuhkan tampilan item baru (belum ada yang bisa digunakan ulang), fungsi ini akan menggunakan LayoutInflater untuk meng-inflate file layout XML item_list.xml menjadi objek tampilan (View), membungkusnya ke dalam objek ItemListAdapter agar komponen UI-nya bisa diakses langsung tanpa findViewById, lalu membentuk dan mengembalikan ViewHolder yang menyimpan view tersebut sebagai wakil satu baris item di dalam list.

Line 21 dan 22, menghubungkan data ke tampilan item yang sudah dibuat oleh RecyclerView, dengan cara mengambil data Item dari list berdasarkan indeks position yang menunjukkan posisi item tersebut di dalam daftar, lalu memanfaatkan objek holder (yang berisi binding tampilan) untuk menampilkan data seperti gambar, judul, dan informasi lain ke tampilan.

Line 23 sampai 26, Men-set gambar, judul, dan penulis ke elemen UI di item_list.xml.

Line 28 sampai 33, Tombol openButton akan menjalankan fungsi onItemClick dengan parameter URL item. Tombol detailButton akan menjalankan fungsi onItemClick dengan parameter seluruh data Item.

Item_list.xml :

Line 2, komponen dari AndroidX yang menampilkan isi dalam bentuk kartu dengan sudut melengkung dan bayangan (elevation). Ideal untuk membuat tampilan item yang rapi dan modern di dalam list.

Line 3 dan 4, Mendefinisikan namespace:

`xmlns:android` digunakan untuk atribut standar Android.

`xmlns:card_view` digunakan untuk atribut khusus CardView, seperti `cardCornerRadius` dan `cardElevation`.

Line 5 dan 6, Ukuran CardView akan mengisi seluruh lebar parent (`match_parent`) dan tingginya menyesuaikan dengan isi kontennya (`wrap_content`).

Line 7, Memberikan jarak ke bawah antar item dalam list, sehingga item tidak terlihat terlalu rapat satu sama lain.

Line 8, Membuat sudut kartu melengkung dengan radius 16dp untuk tampilan lebih lembut dan modern.

Line 9, membuat sudut luar CardView melengkung dengan radius 16dp.

Line 10, memberi efek bayangan agar kartu tampak mengambang (elevated look).

Line 12 sampai 16, Kontainer vertikal yang menyusun isi kartu dari atas ke bawah: gambar, teks, dan tombol. Diberi padding 16dp agar isi tidak menempel ke tepi kartu.

Line 18 sampai 23 komponen gambar dari Material Components.
`shapeAppearanceOverlay="@style/RoundedImageView"` memberi radius pada sudut gambar, membuat tampilannya membulat (rounded corner). Lebar penuh, tinggi tetap 180dp. `scaleType="centerCrop"` menjaga proporsi gambar agar mengisi view secara penuh tanpa distorsi.

Line 25 – 32, menampilkan judul item. `textStyle="bold"` dan `textSize="18sp"` membuatnya menonjol. `paddingTop="8dp"` memberi jarak dari gambar.

Line 34 sampai 38, menampilkan subjudul atau deskripsi singkat. Ditempatkan langsung di bawah judul.

Line 40 sampai 44, menyusun tombol secara horizontal. `gravity="end"` memastikan kedua tombol disejajarkan ke kanan.

Line 46 sampai 50 Tombol untuk membuka sesuatu, bisa diarahkan ke fitur atau aksi utama.

Line 52 sampai 57, tombol untuk menampilkan detail item. Diberi margin kiri 8dp agar tidak terlalu menempel dengan tombol "Open".

HomeFragment.kt :

Line 3 sampai 12, import semua library yang dibutuhkan:

Intent dan Uri: untuk membuka browser menggunakan intent eksplisit.

Fragment: karena class ini merupakan turunan dari Fragment.

findNavController: digunakan untuk navigasi antar fragment.

LinearLayoutManager: digunakan untuk menampilkan list secara vertikal.

FragmentHomeBinding: menghubungkan layout fragment_home.xml dengan kode menggunakan ViewBinding.

Line 14, Deklarasi class fragment utama bernama HomeFragment. Class ini digunakan untuk menampilkan daftar item.

Line 15 dan 16, _binding menyimpan binding layout, nilainya bisa null. binding mengakses _binding dengan jaminan tidak null (!) untuk digunakan di seluruh fragment.

Line 18 sampai 24, Ketika onCreateView dipanggil, LayoutInflater digunakan untuk mengubah layout XML fragment_home.xml menjadi objek tampilan nyata di aplikasi. Proses ini dilakukan melalui FragmentHomeBinding.inflate, yang secara otomatis menghubungkan semua elemen UI dalam layout dengan kode menggunakan ViewBinding. Hasil dari proses ini kemudian dikembalikan melalui binding.root, yaitu tampilan utama (root view (View)) dari fragment tersebut yang akan ditampilkan di layar.

Line 26, Dipanggil setelah onCreateView selesai dan layout sudah siap ditampilkan.

Line 27 sampai 33, Membuat daftar item yang akan ditampilkan di RecyclerView. Setiap item berisi parameter yang diambil dari item.kt yaitu gambar (imageResId), judul (title), penulis (author), dan URL (url).

Line 35 sampai 39, Membuat objek adapter yang diberi itemList sebagai data. onClick digunakan ketika tombol “Open” diklik, akan membuka URL di browser pakai Intent.ACTION_VIEW.

Line 40 sampai 43, onClick digunakan untuk membuka URL milik suatu item ke browser saat tombol “Open” diklik, dengan memanfaatkan Intent.ACTION_VIEW untuk membuat intent eksplisit.

Line 45 sampai 46, binding.recyclerView.layoutManager menentukan pola tampilan item (vertikal), sedangkan binding.recyclerView.adapter menetapkan data dan tampilan item yang akan ditampilkan di RecyclerView menggunakan ItemAdapter

Line 49 sampai 52, Supaya tidak terjadi memory leak, binding di-nol-kan saat tampilan sudah tidak digunakan.

fragment_home.xml :

Line 2 sampai 6, mendefinisikan ConstraintLayout sebagai root layout. ConstraintLayout adalah jenis layout fleksibel yang memungkinkan pengaturan posisi elemen berdasarkan hubungan antar elemen (constraints). xmlns:android dan xmlns:app adalah deklarasi namespace agar atribut Android standar (android:) dan atribut tambahan dari library (app:) dikenali. layout_width="match_parent" dan layout_height="match_parent" membuat layout mengisi seluruh lebar dan tinggi dari layar atau container-nya.

Line 8 sampai 18, mendefinisikan komponen RecyclerView, yaitu elemen tampilan berbasis list yang digunakan untuk menampilkan data dalam jumlah banyak secara efisien.

android:id="@+id/recyclerView" memberikan ID unik agar RecyclerView dapat diakses dari kode Kotlin menggunakan data binding (binding.recyclerView).

android:layout_width="0dp" dan android:layout_height="0dp" digunakan karena dalam ConstraintLayout, ukuran 0dp berarti mengikuti aturan constraint yang diberikan (disebut juga *match constraints*).

android:clipToPadding="false" membuat konten yang ditampilkan oleh RecyclerView tidak dipotong oleh padding, memungkinkan efek visual seperti bayangan atau overscroll.

android:padding="16dp" memberikan jarak 16dp di sekeliling RecyclerView agar item tidak menempel ke tepi layar, meningkatkan kenyamanan visual.

app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager" menentukan jenis layout manager yang digunakan oleh RecyclerView. Dalam hal ini, LinearLayoutManager digunakan untuk menampilkan item secara vertikal satu per satu.

app:layout_constraintTop_toTopOf="parent" dan constraint lainnya mengatur posisi RecyclerView agar menempel ke keempat sisi parent-nya (ConstraintLayout), membuatnya memenuhi seluruh ruang tampilan.

Line 19, menutup elemen ConstraintLayout, menandai akhir dari struktur layout.

nav_graph.xml :

Line 2 sampai 6, <navigation> adalah tag utama untuk mendeklarasikan Navigation Graph.

xmlns:android, xmlns:app, dan xmlns:tools adalah deklarasi namespace yang dibutuhkan untuk atribut-attribut dari Android SDK, Jetpack, dan tools.

android:id="@+id/nav_graph" menetapkan ID untuk graph ini agar bisa direferensikan di kode atau layout lain.

app:startDestination="@id/homeFragment" menyatakan bahwa homeFragment adalah fragment pertama yang ditampilkan ketika aplikasi dibuka.

Line 8 sampai 16, android:id="@+id/homeFragment" memberi ID pada fragment agar bisa direferensikan di graph dan kode.

`android:name="com.example.listxml.HomeFragment"` menunjukkan class fragment yang digunakan.

`android:label="Home"` memberikan label tampilan, biasanya digunakan di toolbar/judul fragment.

`tools:layout="@layout/fragment_home"` adalah preview hint untuk Android Studio saat menampilkan layout preview.

Di dalamnya terdapat tag `<action>`: `android:id="@+id/action_home_to_detail"` adalah ID untuk aksi navigasi dari Home ke Detail. `app:destination="@id/detailFragment"` menentukan tujuan navigasi, yaitu ke `detailFragment`.

Line 18 sampai 32, Menyediakan `DetailFragment` sebagai halaman tujuan dengan parameter atau data yang akan ditampilkan secara dinamis berdasarkan item yang diklik di `HomeFragment`.

Fragment ini akan menerima data dari `HomeFragment` melalui tiga argumen, yaitu:

`title` (tipe string), `subtitle` (tipe string), `imageResId` (tipe integer, biasanya ID resource gambar)

Line 33, Menutup elemen `<navigation>`, yang menandakan akhir dari definisi `Navigation Graph`.

DetailFragment.kt :

Line 11, Mendeklarasikan class `DetailFragment` yang merupakan subclass dari `Fragment`.

Line 12 dan 13, `_binding` adalah variabel nullable yang menyimpan instance binding dari layout `fragment_detail.xml`. `binding` adalah versi non-nullable dari `_binding`, digunakan untuk akses aman ke elemen layout. Teknik ini digunakan untuk menghindari memory leak dengan cara menghapus binding saat view di-destroy.

Line 14, Mendeklarasikan variabel `args` yang otomatis terisi oleh argument yang dikirim dari `HomeFragment`. `DetailFragmentArgs` adalah class yang secara otomatis dibuat oleh `Navigation Component` berdasarkan deklarasi di `nav_graph.xml`

Line 16 sampai 22, `onCreateView()` dipanggil saat fragment akan menampilkan layout-nya. `FragmentDetailBinding.inflate(...)` digunakan untuk menghubungkan layout XML ke fragment menggunakan View Binding. `binding.root` mengembalikan elemen root dari layout (yaitu `ConstraintLayout` dari `fragment_detail.xml`) sebagai tampilan utama fragment.

Line 24 sampai 28, Fungsi ini dipanggil setelah layout fragment selesai dibuat. Tiga baris di dalamnya mengatur isi tampilan berdasarkan argument yang diterima:

`binding.detailTitle.text = args.title`: Menampilkan judul yang dikirim.

`binding.detailSubtitle.text = args.subtitle`: Menampilkan subjudul/penulis.

`binding.detailImage.setImageResource(args.imageResId)`: Menampilkan gambar berdasarkan resource ID.

Line 30 sampai 33, Fungsi ini dipanggil saat tampilan fragment akan dihancurkan. `_binding` di-set ke null untuk menghindari kebocoran memori (memory leak) karena binding terikat dengan view yang sudah tidak ada.

Fragment_detail.xml :

Line 2 sampai 6, ScrollView adalah layout yang memungkinkan konten di dalamnya bisa di-scroll secara vertikal. `xmlns:android` dan `xmlns:app` adalah deklarasi namespace untuk atribut standar dan khusus., diperlukan agar atribut android: dan app: dikenali.

`layout_width="match_parent"` dan `layout_height="match_parent"` membuat ScrollView mengisi seluruh lebar dan tinggi layar. `padding="16dp"` memberi jarak sebesar 16dp di sekeliling konten untuk tampilan yang lebih rapi dan tidak terlalu rapat ke tepi layar.

Line 8 sampai 11, LinearLayout digunakan untuk menata elemen UI secara **vertikal** (atas ke bawah). `layout_width="match_parent"`, mengisi lebar kontainer (yaitu ScrollView). `layout_height="wrap_content"`, tinggi menyesuaikan isi di dalamnya. `orientation="vertical"`, menyusun elemen satu per satu dari atas ke bawah.

Line 13 sampai 18, ShapeableImageView digunakan untuk menampilkan gambar dari item yang dipilih. `@+id/detailImage` adalah ID yang digunakan untuk mengakses elemen ini di Kotlin. `layout_width="match_parent"`, gambar mengisi seluruh lebar layar. `layout_height="200dp"`, tinggi gambar tetap, yaitu 200dp. `scaleType="centerCrop"`, gambar dipotong dan diperbesar agar mengisi seluruh area, menjaga proporsi sambil menampilkan bagian tengah gambar. `app:shapeAppearanceOverlay="@style/RoundedImageView"`: memberi efek sudut melengkung (rounded corner) pada gambar

Line 20 sampai 26, TextView ini digunakan untuk menampilkan judul item. `wrap_content`, Lebar dan tinggi menyesuaikan panjang teks. `textStyle="bold"`, Menjadikan teks tebal (bold). `textSize="20sp"`, Ukuran huruf sebesar 20sp. `paddingTop="8dp"`, jarak atas antara gambar dan teks sebesar 8dp agar tidak terlalu rapat.

Line 28 sampai 31, TextView ini digunakan untuk menampilkan subjudul atau nama penulis/penerbit item. `wrap_content`: Ukuran elemen akan menyesuaikan isi teks. Tidak ada atribut gaya tambahan karena ini hanya teks penjelas.

MainActivity.kt :

Line 2, mengimpor kelas Bundle, yang digunakan untuk menyimpan data sementara, terutama saat menyimpan atau memulihkan status aktivitas (Activity).

Line 3, mengimpor kelas AppCompatActivity, yaitu kelas dasar (superclass) untuk semua aktivitas yang mendukung fitur-fitur modern Android melalui AppCompatActivity library, seperti tema Material Design dan fragment.

Line 4, mengimpor kelas `ActivityMainBinding`, yaitu kelas otomatis yang dihasilkan oleh fitur `ViewBinding` berdasarkan file `activity_main.xml`. Ini memungkinkan kita mengakses langsung komponen UI dalam layout tanpa perlu `findViewById()`.

Line 7, mendefinisikan kelas `MainActivity` yang **mewarisi** (extends) `AppCompatActivity`. `MainActivity` berfungsi sebagai aktivitas utama (main screen) aplikasi.

Line 8, mendeklarasikan variabel binding dengan tipe `ActivityMainBinding`. `lateinit` artinya variabel akan diinisialisasi nanti, bukan saat deklarasi. Variabel ini akan digunakan untuk mengakses elemen UI yang didefinisikan dalam `activity_main.xml`.

Line 10, `onCreate()` adalah fungsi siklus hidup aktivitas yang pertama kali dipanggil saat aktivitas dibuat. Parameter `savedInstanceState` menyimpan status sebelumnya jika aktivitas pernah ditutup (misalnya karena rotasi layar).

Line 11, memanggil implementasi `onCreate()` dari superclass (`AppCompatActivity`) untuk memastikan bahwa semua proses internal Android dilakukan dengan benar sebelum melanjutkan.

Line 12, menginisialisasi binding menggunakan `inflate(layoutInflater)`, yang berarti sistem akan membuat view dari file `activity_main.xml`. Dengan ini, kita bisa mengakses elemen-elemen layout menggunakan nama ID-nya dalam kode Kotlin (contoh: `binding.textViewTitle`).

Line 13, Menetapkan tampilan utama (view) dari aktivitas menggunakan root view dari binding. `binding.root` adalah root layout dari `activity_main.xml`, yang akan ditampilkan di layar.

activity_main.xml :

Line 2, merupakan *view container* modern yang digunakan untuk menampilkan fragment. Ini adalah pengganti dari `<fragment>` di versi Android yang lebih baru. Berasal dari library `androidx.fragment.app`, dan memberikan kontrol yang lebih baik dalam menampilkan dan mengganti fragment secara dinamis.

Line 3 dan 4, `xmlns:android` dan `xmlns:app` adalah deklarasi namespace XML: `xmlns:android` digunakan untuk atribut standar Android (misalnya: `layout_width`, `id`). `xmlns:app` digunakan untuk atribut khusus AndroidX atau Jetpack (misalnya: `app:navGraph`).

Line 5, memberi ID `nav_host_fragment` pada `FragmentContainerView`. ID ini penting karena akan digunakan oleh kode Kotlin/Java atau XML lain untuk merujuk ke view ini, khususnya oleh `NavController`.

Line 6, menentukan bahwa container ini akan menampung **`NavHostFragment`**, yaitu fragment khusus dari Jetpack Navigation yang bertugas menjadi **penyedia navigasi**.

NavHostFragment akan menjadi tempat semua fragment lain (seperti HomeFragment, DetailFragment) ditampilkan sesuai navigasi.

Line 7 dan 8, `match_parent` artinya lebar dan tinggi container akan menyesuaikan ukuran parent-nya, yaitu Activity.

Line 9, menentukan file navigasi graph (`nav_graph.xml`) yang digunakan oleh NavHostFragment. File ini berisi daftar fragment, arah navigasi antar fragment, dan data/argumen yang diperlukan saat berpindah.

Line 10, menandakan bahwa NavHostFragment ini adalah host navigasi utama dalam aplikasi. artinya, ketika pengguna menekan tombol back, sistem akan meminta navigasi kembali ke fragment sebelumnya di stack navigasi ini. hanya boleh ada satu `defaultNavHost="true"` di setiap activity.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/NazmiHakim/Pemrograman-Mobile/tree/main/Modul3>

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Pembahasan

RecyclerView masih banyak digunakan karena merupakan bagian dari View System yang telah lama ada dan stabil, serta kompatibel dengan berbagai library dan arsitektur Android lama. Meskipun penulisannya cenderung panjang dan boiler-plate, RecyclerView sangat fleksibel dan mendukung berbagai jenis layout serta interaksi kompleks seperti drag-and-drop dan item animasi khusus. Selain itu, banyak developer dan perusahaan besar sudah terbiasa serta memiliki infrastruktur berbasis RecyclerView, sehingga migrasi ke Compose memerlukan pertimbangan matang. LazyColumn dari Jetpack Compose memang menawarkan kode yang lebih ringkas, namun belum sepenuhnya menggantikan fleksibilitas dan kestabilan yang ditawarkan RecyclerView dalam kasus-kasus tertentu.